Week 4-Part 1
Logistic Regression

Course: Data 201

Logistic Regression

From **R (glm, tidymodels)** to **Python (statsmodels, scikit-learn)**

FeatureLinear RegressionLogistic Regression

Goal	Predict a **continuous value**	Predict a **probability / category**
Output	Any real number	Between **0 and 1**
Example	Predict **house price**	Predict **survival (yes/no)**
Model form	$y = \beta_0 + \beta_1 x$	$P = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$
Graph shape	Straight line	**S-shaped curve**
Interpretation	Change in **value**	Change in **probability**

Learning Goals

By the end of today, you should be able to:

- Explain logistic regression using **log-odds** and **odds ratios**
- Translate an R glm() model into Python using statsmodels
- Translate a tidymodels workflow into scikit-learn
- Choose modeling tools based on **inference vs prediction** goals
- Evaluate classification models appropriately

Where This Fits in the Course

So far you've learned:

- Linear regression (inference vs prediction)
- Python modeling ecosystem

Today's focus:

➡ **Binary outcomes + workflow choices**

Logistic Regression: Same Model Everywhere

Key idea:

Logistic regression is the **same statistical model** in R and Python.

Differences arise from: - Defaults - Workflows - Goals (inference vs prediction)

Logistic Regression in R: glm

Typical R workflow:

```r
glm(y ~ x1 + x2,
    data = df,
    family = binomial)
```

Python equivalent (statsmodels):

```python
import statsmodels.formula.api as smf

model = smf.logit("y ~ x1 + x2", data=df).fit()
print(model.summary())
```

Characteristics:

- Inference-first
- Formula interface
- Coefficients, SEs, p-values

Logistic Regression in R: tidymodels

Modern R workflow:

```r
library(tidymodels)

logistic_reg() %>%
  set_engine("glm") %>%
  set_mode("classification")
```

Python equivalent (scikit-learn-style workflow):

```python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
```

Characteristics:

- Workflow-first
- Explicit resampling
- Metrics-focused

Logistic Regression in Python

statsmodels

- Focus: **statistical interpretation**
- Provides **model summary and p-values**
- Uses **R-style formula syntax**

scikit-learn

- Focus: **prediction and machine learning**
- Used for **classification tasks**
- Works with **ML workflows**

Python Mirrors This Split

Python does *not* mix these by default:

R	Python
glm	statsmodels
tidymodels	scikit-learn

➡ Same ideas, different libraries

statsmodels ≈ glm

Python inference workflow:

```python
import statsmodels.formula.api as smf

model = smf.logit("y ~ x1 + x2", data=df).fit()
```

Produces:

- Coefficients
- Standard errors
- p-values

Interpreting Logistic Regression Output

Coefficients

- Show the **effect of each feature** on the outcome
- Positive → increases probability of (y=1)
- Negative → decreases probability of (y=1)

Standard Errors

- Measure the **uncertainty of the coefficient estimate**
- Smaller value → more reliable estimate

p-values

- Test whether a variable has a **statistically significant effect**
- Small p-value (often < 0.05) → variable likely important

Interpreting Coefficients

Important reminders:

- Coefficients are **log-odds**
- Exponentiating gives **odds ratios**

```python
import numpy as np

np.exp(model.params)
```

Same interpretation as R

scikit-learn ≈ tidymodels

Prediction-focused workflow:

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

Key differences:

- No formulas
- Regularization by default (tunable with `penalty` and `C`)

- Regularization is a **built-in "penalty" that discourages the model from using huge coefficients**.
- (typically L2) to reduce overfitting by shrinking coefficients.
- **`penalty`** specifies the type of regularization (`l2`, `l1`, or `none`).
- **`C`** controls the strength of regularization:
  - smaller `C` → stronger regularization
  - larger `C` → weaker regularization

What happens during `fit()`

- The model **estimates the coefficients** (\beta_0, \beta_1, \beta_2)
- Uses **Maximum Likelihood Estimation (MLE)**
- Goal: find parameters that **maximize the likelihood of the observed data**
- Solved using an **iterative optimization algorithm** (e.g., LBFGS)

Key idea

Training the model = **estimating parameters from the data**

Iterative Optimization in Logistic Regression

- Logistic regression parameters are estimated using **Maximum Likelihood Estimation (MLE)**.
- There is **no closed-form solution**, so we use **iterative optimization algorithms**.

Idea of iterative optimization

- Start with **initial guesses** for the coefficients.
- Repeatedly **update the coefficients** to improve the likelihood.
- Stop when the solution **converges** (no significant improvement).

Newton's Method

- A common iterative optimization method.
- Uses information from:
  - **First derivative (gradient)** → slope
  - **Second derivative (Hessian)** → curvature
- Updates parameters step by step until the **maximum likelihood** is found.

Probabilities, Not Just Classes

Why probabilities matter:

- Flexible thresholds
- Risk-based decisions
- Better evaluation

```
model.predict_proba(X)
```

Probabilities, Not Just Classes

Logistic regression predicts **probabilities**, not just 0 or 1.

Why probabilities matter

- **Flexible thresholds** (not always 0.5)
- **Risk-based decisions**
- **Better model evaluation** (ROC, AUC)

Risk-Based Decisions

Prediction errors are not always equally serious.

Two common types of mistakes:

- **False positive** → predict event when it did not happen
- **False negative** → miss the event when it actually happened

Why This Matters

Some mistakes have **higher risk or cost**.

Example: Medical diagnosis

Prediction	Reality	Result
No disease	Has disease	Dangerous (missed diagnosis)
Has disease	No disease	Extra testing

Idea

Adjust the **probability threshold** depending on the **cost of errors**.

Better Model Evaluation (ROC, AUC)

Logistic regression produces **probabilities**, which allow better ways to evaluate the model.

ROC Curve

- Plots **True Positive Rate** vs **False Positive Rate**
- Shows model performance across **different thresholds**

AUC (Area Under the Curve)

- Measures **overall model performance**
- **Closer to 1 → better model**
- **0.5 → random guessing**

Model Evaluation

Common metrics:

- Accuracy (threshold-dependent)
- Confusion matrix
- ROC curve
- AUC (threshold-independent)

➡ Match metric to goal

Choosing the Right Tool

Ask first:

- Do I want **explanation** or **prediction**?
- Do I need uncertainty estimates?
- Will this model be deployed?

Then choose: - statsmodels / glm - scikit-learn / tidymodels

Active Learning (15–20 min)

Explain, Compare, Decide

1. Explain logistic regression using **odds**, not probabilities
2. Compare:
    - glm vs statsmodels
    - tidymodels vs scikit-learn
3. Decide which tool you'd use for:
    - Policy analysis
    - Price prediction

➡ See Student Notebook

Common Pitfalls

- Interpreting odds ratios as probability changes
- Using accuracy alone
- Ignoring class imbalance
- Forgetting regularization defaults

Key Takeaways

- Logistic regression is conceptually unchanged
- Python makes workflow choices explicit
- statsmodels ≈ glm
- scikit-learn ≈ tidymodels

Looking Ahead

Next topics:

- Threshold tuning
- Imbalanced data
- Regularized logistic regression
- Model calibration

Thanks